

03 · NAND 완전이해 — 전하 한 무리를 절연막에 가두는 일

이 문서의 약속 이건 요약이 아니라 정독용 해설서다. NAND를 "floating gate에 전자를 가둔다"는 한 줄로 외우는 대신, 왜 절연막에 전자를 가두면 데이터가 되는지, 왜 program은 page 단위인데 erase는 block 단위인지, 왜 2D를 버리고 굳이 위로 쌓았는지를 1차 원리에서 끝까지 따라간다. 너는 삼성 S.LSI 로직 디지털설계/검증 4년차다. 셀 어레이의 물리는 낫설지만, NAND의 절반 — 컨트롤러(FTL)와 Peri 로직 — 은 네가 4년간 한 일과 같은 종류의 문제다. 그 다리를 끝에서 놓는다. 모든 변동/시장 수치는 (추정 · 변동) 으로 표기한다. 내부 비공개 수치는 쓰지 않는다.

0. 한 장면에서 시작하자 — "전원을 뽑아도 살아남는 메모리"

DRAM을 먼저 떠올리자. DRAM 셀은 커패시터에 전하를 담는다. 커패시터는 본질적으로 새는 양동이다. 전자가 누설로 빠져나가므로 64ms(통상값, 변동)마다 읽어서 다시 채워줘야 한다(refresh). 전원을 뽑으면? 양동이는 순식간에 비고, 데이터는 사라진다. 이게 휘발성이다.

이제 질문을 뒤집자. 전원을 뽑아도 몇 년씩 데이터가 살아남으려면, 전하를 담은 그릇이 "새지 않아야" 한다. 새지 않는 그릇이란 곧 사방이 절연막으로 둘러싸인 전하 저장소다. 절연막(주로 SiO_2 , 산화막)은 전자가 통과하기 매우 어려운 벽이다. 그 벽 안쪽에 전하를 한 무리 가둬두면, 평상시엔 절대 빠져나오지 못하고 — 그래서 비휘발성이고 — 다만 우리가 원할 때(매우 높은 전압을 걸 때)만 그 벽을 "터널링"으로 넘게 만든다.

이 한 문장이 NAND 전체의 씨앗이다.

NAND = "절연막에 갇힌 전하 한 무리"가 트랜지스터의 문턱전압(V_{th})을 밀어, 그 V_{th} 가 1이냐 0이냐를 말한다.

여기서부터 모든 것이 따라 나온다. 비휘발성도, 고밀도도, 느린 쓰기도, 마모도, 3D도, ECC도. 차근차근 가자.

0.1 왜 NAND가 "스토리지"이고 DRAM이 "메모리"인가

컴퓨터에는 두 종류의 기억이 필요하다.

- **작업 기억(working memory)**: CPU가 지금 계산에 쓰는 데이터. 빠르게, 아무 주소나(random access) 즉시 읽고 써야 한다. 전원 끄면 사라져도 된다(어차피 작업이 끝났으니). → **DRAM**.
- **영구 저장(storage)**: 전원을 꺼도 남아야 하는 데이터(OS, 사진, 모델 weight). 다소 느려도 되고, 한 번에 큰 덩어리로 읽고 쓴다. 대신 싸고, 뽁뽁하고, 안 사라져야 한다. → **NAND**.

이 역할 분담은 우연이 아니라 물리의 귀결이다. DRAM은 커패시터라는 단순한 그릇 덕에 빠르고 무한히 다시 쓸 수 있지만 휘발성이고 비싸다(셀마다 커패시터 자리가 필요). NAND는 전하를 절연막에 가두느라 쓰기가 느리고 셀이 마모되지만, 그 대가로 비휘발성 + 극단적 고밀도 + 최저 비용/bit를 얻는다.

비유하자면 DRAM은 책상 위 메모지(빠르게 적고 지우지만 바람 불면 날아감), NAND는 잉크로 적어 서랍에 넣는 장부(적는 데 시간 걸리고 종이가 닳지만 몇 년 뒤에도 읽힘)다.

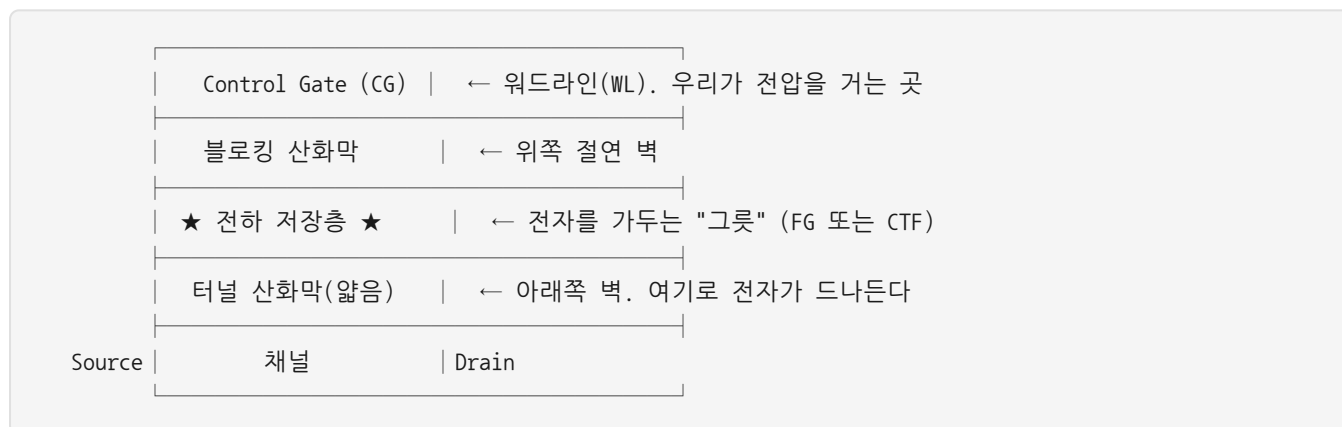
면접에서 "DRAM과 NAND의 근본 차이"(★A1)를 물으면, 셀 구조 나열로 끝내지 말고 **"역할이 달라서 물리를 다르게 골랐다"**는 인과로 답해야 깊다. 표는 §11에 모았다.

1. 전하 저장의 물리 — Floating Gate vs Charge Trap

1.1 보통 MOSFET을 떠올리고, 그 위에 그릇을 하나 얹자

너는 MOSFET을 안다(★B2). 게이트에 전압을 걸면 gate-oxide 커패시터를 통해 채널에 전하가 유도되고, 문턱전압 V_{th} 를 넘으면 inversion layer(채널)가 생겨 source-drain 사이에 전류가 흐른다. V_{th} 는 "이 트랜지스터를 켜려면 게이트에 최소 얼마를 걸어야 하는가"라는 문턱값이다.

NAND 셀은 이 평범한 트랜지스터의 게이트와 채널 사이에 전하 저장층을 하나 끼워 넣은 것이다. 구조를 아래에서 위로 쌓아 보면:



핵심 아이디어: 저장층에 전자가 얼마나 들어 있느냐가, 이 트랜지스터를 "겉에서 본" V_{th} 를 바꾼다.

왜? 저장층에 음전하(전자)가 모여 있으면, 그 음전하가 control gate가 채널에 만들려는 전기장을 일부 상쇄(screen)한다. 그래서 채널을 켜려면 control gate에 더 높은 전압을 줘야 한다 → V_{th} 가 올라간다. 반대로 저장층이 비어 있으면 control gate의 영향이 그대로 채널에 전달돼 V_{th} 가 낮다.

저장층에 전자 가득 = V_{th} 높음 = 관례상 "0"(programmed) 저장층 비어 있음 = V_{th} 낮음 = 관례상 "1" (erased)

그래서 데이터를 읽는다는 건 결국 "이 셀의 V_{th} 가 높은가 낮은가"를 판정하는 일이다. 데이터를 쓴다는 건 저장층의 전자 수를 조절해 V_{th} 를 원하는 위치로 옮기는 일이다. 외워야 할 단 하나의 문장: NAND의 모든 동작은 V_{th} 를 옮기고(write/erase) 읽는(read) 일이다.

1.2 비휘발성의 물리적 이유 — 왜 안 사라지는가

저장층은 위아래가 모두 절연막(블로킹 산화막 + 터널 산화막)으로 둘러싸여 있다. 절연막은 전자에게 높은 에너지 장벽(potential barrier)이다. 상온에서 전자가 이 장벽을 열적으로 넘을 확률은 천문학적으로 낮다. 그래서 한 번

가둔 전자는 — 외부에서 강한 전기장을 걸어 억지로 빼내지 않는 한 — 수년~수십 년 그 자리에 머문다. 이것이 비휘발성의 정체다.

대조적으로 DRAM 커패시터는 절연이 그만큼 완벽하지 않고(누설 경로가 있고), 또 그게 의도다(빠른 access를 위해 단순한 구조를 택했으니까). NAND는 정반대 trade-off — **느려도 좋으니 절대 안 새게** — 를 택했다. 같은 "전하를 담는다"는 행위지만 **그릇의 절연 품질을 정반대로 설계한 것이** 두 메모리의 분기점이다.

다만 "절대 안 샌다"는 이상화일 뿐, 시간이 흐르면 아주 조금씩 전자가 빠져나간다(retention 저하). 그리고 셀을 많이 쓰면 절연막이 닳아 새기 시작한다(\$9의 수명 이야기). 이 누설을 디지털로 덮는 게 ECC와 read reclaim이다 — 미리 적어두자.

1.3 Floating Gate — 도체 그릇

전하 저장층을 무엇으로 만드느냐에 따라 두 갈래로 나뉜다. 역사적으로 먼저 온 것이 **Floating Gate(FG)**다.

FG는 저장층을 **도체(전도성 폴리실리콘)**로 만든다. "floating(떠 있는)"이라는 이름은 이 도체 게이트가 어디에도 전기적으로 연결되지 않고 절연막에 동동 떠 있다는 뜻이다. 전자를 주입하면 도체이므로 전하가 **저장층 전체에 자유롭게 퍼져** 균일하게 분포한다.

- **장점:** 전하가 잘 퍼져 안정적이고, 오래 검증된 성숙한 기술.
- **결정적 약점 1 — 셀 간 간섭(coupling):** 도체 덩어리는 옆 셀의 도체 덩어리와 전기적으로 결합한다. 셀이 뽁뽁해질수록 옆 셀의 전하 상태가 내 V_{th} 에 영향을 준다(floating gate coupling). 미세화의 적이다.
- **결정적 약점 2 — 단일 결합에 취약:** 도체라서, 절연막 한 군데에 결합(구멍)이 생기면 **저장된 전하 전체가 그 구멍으로 한꺼번에 새어 나간다**. 양동이에 구멍 하나 = 전체 누수.

이 두 약점이 셀을 미세화하고 3D로 쌓으려 할 때 치명적이 된다.

1.4 Charge Trap(CTF) — 절연체 그릇, 그리고 왜 3D는 이걸 택했나

Charge Trap Flash(CTF)는 저장층을 **절연체(질화막, Si_3N_4)**로 만든다. 질화막 안에는 미시적인 "trap site(전자 덩)"들이 무수히 흩어져 있고, 전자는 거기에 **하나하나 이산적으로(discretely) 포획**된다. 도체처럼 자유롭게 퍼지지 않고, 잡힌 자리에 머문다.

이 한 가지 차이 — **연속(도체) vs 이산(절연체)** — 가 모든 장점을 만든다.

- **셀 간 간섭↓:** 전하가 이산적으로 고정돼 있어 옆 셀과의 전기적 결합이 작다. 뽁뽁하게 쌓아도 덜 간섭한다.
- **단일 결합에 강함:** 절연막에 구멍 하나가 나도, 전자들이 도체처럼 한꺼번에 흘러나가지 않는다. **구멍 근처의 전자 몇 개만** 잃는다. 양동이가 아니라 스펀지여서, 한 군데 찢어져도 전체가 쏟아지지 않는다.
- **구조가 단순하고 얇음:** 도체 FG보다 얇게 만들 수 있어 **수직 적층(3D)에 결정적으로 유리**하다. channel hole 벽을 따라 얇은 막으로 두르기에 적합하다(\$7).

그래서 **현대 3D NAND는 대부분 CTF**를 쓴다. SK하이닉스를 포함한 주요 업체의 3D V-NAND가 CTF 기반이다.

- **CTF의 단점도 정직하게:** 전하가 이산적이라 시간에 따른 산포(retention 산포)·프로그램 산포 관리가 과제다. trap에 잡힌 전자가 옆 trap으로 이동하거나(lateral migration) 시간이 지나며 분포가 흐트러질 수 있다. 그래서 CTF라고 신뢰성이 공짜는 아니고, 여전히 ECC-refresh가 받쳐준다.

면접 골격(Q1 꼬리): "FG는 도체라 전하가 연속·셀 간섭 큼·결함 하나에 전체 누수. CTF는 절연체 trap에 이산 포획이라 간섭↓·국소 결함에 강함·얇아서 3D 적층 유리. 그래서 현대 3D NAND는 CTF." — 인과를 한 호흡에.

2. 읽기(Read) — Vth를 어떻게 판정하나

이제 데이터가 Vth로 저장돼 있다는 걸 안다. 읽기는 "이 셀의 Vth가 기준보다 높은가 낮은가"를 알아내는 일이다. 방법은 우아하다.

선택한 셀의 워드라인(WL = control gate)에 read 판정 전압 Vread를 건다. Vread는 "높은 Vth"와 "낮은 Vth"의 사이값으로 고른다. 그러면:

- 셀의 Vth가 Vread보다 낮으면(저장층 비어 있음, "1") → 셀이 켜진다(ON) → 전류가 흐른다.
- 셀의 Vth가 Vread보다 높으면(저장층에 전자 가득, "0") → Vread로는 문턱을 못 넘어 셀이 꺼진 채(OFF) → 전류가 안 흐른다.

즉 "같은 Vread를 걸었을 때 셀이 켜지냐 꺼지냐"가 곧 0/1의 답이다. 이 전류 흐름 여부를 비트라인(BL) 끝의 page buffer / sense amp가 감지해 디지털 값으로 바꾼다. (DRAM의 sense amp와 역할은 닮았지만, DRAM은 미세 전압차를 증폭하고 NAND는 "전류가 흐르냐 안 흐르냐"를 본다.)

2.1 pass 전압(Vpass) — 비선택 셀을 "통로"로 만들기

여기서 NAND 특유의 사정이 끼어든다. 뒤에서 자세히 보겠지만(\$6) NAND는 여러 셀이 하나의 string으로 직렬 연결돼 있다. 우리가 읽으려는 셀 하나만 통과해서는 전류를 측정할 수 없다 — 전류는 string 전체를 관통해야 BL까지 도달한다. 직렬로 연결된 다른(비선택) 셀들이 중간에 OFF면 통로가 막힌다.

그래서 읽을 셀의 WL에만 판정용 Vread를 걸고, 같은 string의 나머지 비선택 셀 WL에는 충분히 높은 pass 전압(Vpass)을 건다. Vpass는 그 셀이 어떤 상태(0이든 1이든, Vth가 높든 낮든)여도 무조건 켜지게 만드는 전압이다. 비선택 셀들을 "무조건 열린 통로"로 만들어 놓고, 오직 선택 셀의 Vread 판정 결과만이 전류 흐름을 좌우하게 하는 것이다.

비유: 긴 복도에 문이 여러 개 직렬로 있다. 어느 한 문(선택 셀)이 잠겼는지 보려면, 나머지 모든 문을 활짝 열어두고(Vpass) 그 문 하나만 살짝 밀어본다(Vread). 끝까지 바람이 통하면 그 문은 열려 있던 것(Vth 낮음, "1"), 안 통하면 잠긴 것(Vth 높음, "0").

이 Vpass가 바로 read disturb의 씨앗이다(\$9). 비선택 셀에 반복적으로 Vpass를 걸면 그 셀에 약한 전하가 야금야금 주입돼 Vth가 슬슬 올라가기 때문이다. 읽기만 했는데 데이터가 망가지는 역설 — 그 뿌리가 여기 있다.

3. 쓰기(Program) — FN 터널링으로 전자 주입, 그리고 ISPP

3.1 FN 터널링 — 절연 벽을 억지로 넘기

절연막은 전자에게 벽이다. 하지만 **충분히 강한 전기장**을 걸면, 양자역학적으로 전자가 그 벽을 **뚫고 통과(터널링)** 할 확률이 급격히 올라간다. 이를 **Fowler-Nordheim(FN) 터널링**이라 한다. 핵심은 "강한 전계 → 얇은 터널 산화막을 전자가 양자 터널링으로 통과"다.

Program(쓰기): 선택 셀의 control gate(WL)에 **매우 높은 program 전압 V_{pgm}** (통상 20V급, 변동)을 걸고 채널은 0V로 둔다. 그러면 control gate ↔ 채널 사이에 강한 전기장이 생기고, **채널의 전자가 터널 산화막을 FN 터널링으로 통과해 저장층으로 주입**된다. 저장층에 전자가 쌓이니 V_{th} 가 올라간다 → "0" 이 된다.

3.2 왜 program은 한 방향(높은 V_{th} , "0")으로만 가는가

이게 면접에서 의외로 깊은 포인트다. **Program은 셀의 V_{th} 를 올리는 동작**뿐이다. 즉 "1(낮은 V_{th})" → "0(높은 V_{th})"으로만 간다. 전자를 주입하는 방향만 program이라 부른다.

그럼 "0"을 "1"로 되돌리려면? 저장층의 전자를 **빼내야** 하고, 그게 바로 **erase**다. 그런데 — 곧 보겠지만 — **erase는 개별 셀이 아니라 block 전체에 대해서만** 할 수 있다. 따라서:

개별 셀/page는 한 방향(1→0)으로만 program 할 수 있고, 되돌리려면(0→1) block 전체를 erase 해야 한다.

이게 NAND가 "제자리 덮어쓰기(in-place update) 불가"인 근본 이유다. 이미 "0"으로 program된 비트를 그 자리에서 다시 "1"로 바꿀 수 없다(셀 단위 erase가 없으니까). 그래서 데이터를 수정하려면 **새 page에 새로 쓰고, 옛 데이터는 나중에 block 통째로 erase할 때 비운다**. 이 비대칭이 컨트롤러(FTL)의 garbage collection·wear leveling이 존재하는 근원이다(§10). 이 한 문단이 NAND 시스템 설계 전체를 떠받친다.

3.3 ISPP — 한 방에 못 쓰는 이유와 증분 펄스

전자를 한 번의 큰 펄스로 왕창 주입하면 안 될까? 안 된다. V_{th} 를 **정밀하게** 원하는 위치에 놓아야 하기 때문이다. 특히 셀당 여러 비트를 저장하는 MLC/TLC/QLC(§8)에서는 V_{th} 분포를 좁고 정확하게 만들어야 인접 레벨과 안 겹친다. 한 방에 세게 주입하면 과하게 올라가 버리고(over-program), 저장층 전자는 셀마다 산포가 있어 같은 펄스에도 셀마다 다르게 반응한다.

그래서 **ISPP(Incremental Step Pulse Programming, 증분 스텝 펄스 프로그래밍)**를 쓴다:

- 작은 V_{pgm} 펄스를 한 번 가한다(전자 조금 주입, V_{th} 조금 상승).
- Verify read**: 이 셀의 V_{th} 가 목표를 넘었는지 즉시 읽어 확인한다.
- 아직 모자라면 V_{pgm} 을 **한 스텝(ΔV_{pgm})** 올려 다시 펄스. 넘었으면 그 셀은 더 이상 주입 금지(program inhibit).
- 모든 셀이 목표 도달할 때까지 program ↔ verify를 반복.

비유: 다트를 한 번에 세게 던지지 않고, **조금씩 던져 과녁에 닿았는지 확인하고, 닿은 다트는 멈추고, 안 닿은 것만 또 던진다**. 셀마다 반응이 달라도 결국 모두 좁은 V_{th} 분포 안에 모인다.

ISPP는 program이 read·erase보다 느리고, bit/cell이 늘수록(QLC) 더 많은 step·verify가 필요해 더 느려지는 이유이기도 하다. **속도-정밀도 trade-off**의 핵심 메커니즘.

4. 지우기(Erase) — block 단위로 전자를 토해내기

Erase는 program의 역방향이다. 저장층의 전자를 **빼내** V_{th} 를 낮춰 "1"로 만든다. 방법은 program과 전계 방향을 뒤집는 것: control gate를 낮게(0V), **채널(또는 기판/bulk)**을 매우 높은 **erase 전압**으로 걸어, 저장층의 전자가 FN 터널링으로 **채널 쪽으로 빠져나가게** 한다.

핵심은 **erase가 block 전체에 동시에 일어난다**는 것이다. 왜 block 단위인지는 §5에서 string 구조로 설명한다 (이게 ★D4·A1의 핵심 꼬리). 여기서는 일단: erase는 "이 block의 모든 셀을 한꺼번에 1로 초기화"하는, 가장 큰 입자(granularity)의 동작이라고 기억하자.

erase 후 block은 깨끗한 백지(전부 "1")가 되고, 그 위에 page 단위로 program(1→0)을 채워 나간다.

5. ★ 왜 read/program은 page 단위인데 erase는 block 단위인가

이건 NAND 면접의 단골이자 깊이 측정기다(★A1·D4). 정답의 핵심은 "**NAND string의 직렬 구조가 강제하는 물리적 귀결**"이다. 외워서가 아니라 구조에서 끌어내자.

5.1 단위(granularity)의 3층 정리

- **셀(cell)**: 1개 트랜지스터 = 최소 저장 단위(1~4비트).
- **page**: 같은 워드라인(WL)에 연결된 셀들의 묶음. **read/program의 단위**(수 KB~수십 KB).
- **block**: 여러 page가 string으로 엮인 묶음. **erase의 단위**(수 MB).

5.2 왜 read/program은 page(=WL) 단위인가

읽기·쓰기는 워드라인 단위로 전압을 거는 동작이다. read는 한 WL에 V_{read} 를, program은 한 WL에 V_{pgm} 을 건다. 하나의 WL은 그 행(row)에 매달린 모든 셀(= 1 page)을 동시에 건드린다. **WL 하나에 전압을 거는 것이 곧 page 하나를 다루는 것** — 그래서 read/program의 자연스러운 단위가 page다. 셀 하나만 읽거나 쓰는 게 아니라, 전압을 공유하는 행 전체가 함께 움직인다.

5.3 왜 erase는 block 단위인가 — string 구조의 귀결

erase는 program과 달리 **채널/기판 쪽에 높은 전압을 걸어 전자를 토해내게** 하는 동작이다. 그런데 NAND에서 **채널(기판)은 개별 셀이나 개별 WL로 쪼개져 있지 않고, block 단위로 공유된다**. 한 block 안의 모든 string이 같은 well/채널 구조를 공유한다.

따라서 erase 전압을 그 공유된 채널/well에 거는 순간, **그 block의 모든 셀이 동시에 전자를 빼앗긴다**. 특정 셀이나 특정 page만 골라 erase할 물리적 경로가 없다 — well은 공유돼 있고, erase 전압은 거기에 걸리니까. 이것이 "erase = block 단위"의 진짜 이유다. 설계 편의가 아니라 **구조가 강제한 결과**다.

한 문장 정답: "read/program은 워드라인에 전압을 거는 동작이라 WL 하나 = page 단위로 가능하지만, erase는 셀들이 공유하는 채널/well에 전압을 걸어 전자를 빼내는 동작이라, well을 공유하는 block 전체가 한꺼번에 지워질 수밖에 없습니다."

5.4 시스템에 주는 파장 — 여기서 FTL이 태어난다

이 비대칭(쓰기는 작게 page, 지우기는 크게 block)이 만든 연쇄:

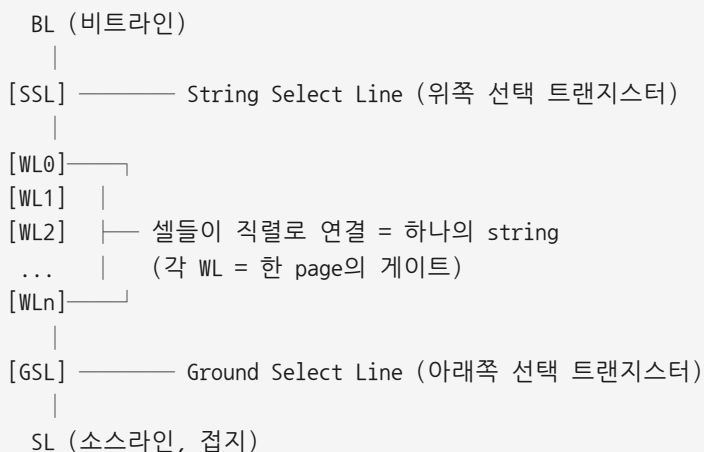
1. **제자리 덮어쓰기 불가**: page 안의 비트 하나만 1로 되돌리려 해도 셀 단위 erase가 없어 불가. → 새 page에 쓰고 옛 자리는 무효(invalid) 표시.
2. **Garbage Collection 필요**: invalid page가 쌓이면, valid page만 새 block으로 옮기고 옛 block을 통째로 erase해 공간 회수.
3. **Write Amplification**: "데이터 1을 수정" → 실제로는 "여러 page 이동 + block erase"가 일어나 *물리적 쓰기량* > *논리적 쓰기량*. 이게 곧 추가 마모.
4. **Over-provisioning · Wear leveling**: 여유 공간을 뒤 GC를 매끄럽게, 쓰기를 전체 block에 고르게 분산해 특정 block 조기 사망 방지.

브리지 선(先)예고: 위 1~4는 전부 컨트롤러(FTL)의 디지털 알고리즘/펌웨어다. 하드웨어의 비대칭 제약을 소프트웨어/로직 추상화로 가리는 것 — 네가 로직에서 늘 하던 "하위 제약을 상위 계층이 흡수"하는 사고와 동형이다. §10·§12에서 달는다.

6. 구조 — NAND string, SSL/GSL, 그리고 직렬의 대가

6.1 왜 "NAND"라는 이름인가 — 직렬 연결

NAND라는 이름은 논리 게이트 NAND에서 왔다. 셀들을 **직렬로 줄줄이 연결**한 모양이 NAND 게이트의 직렬 트랜지스터 배치를 닮아서다. (대비되는 NOR Flash는 셀을 병렬로 두어 random access가 빠르지만 밀도가 낮다.)



- **String**: SSL부터 GSL까지 수직(3D)-직렬로 연결된 셀들의 한 줄. 한 string 안에 수십~수백 개 셀(= 수십~수백 page의 한 조각씩).

- **SSL(String Select Line)**: string의 위쪽 끝 선택 트랜지스터. 이 string을 BL에 연결할지 끊을지 결정.
- **GSL(Ground Select Line)**: string의 아래쪽 끝 선택 트랜지스터. 이 string을 소스라인(접지)에 연결할지 결정.
- 읽을 때: SSL·GSL을 켜서 string을 BL↔SL 사이에 연결하고, 선택 WL엔 Vread, 나머지 WL엔 Vpass. 전류가 string을 관통하면 page buffer가 감지.

6.2 왜 직렬이 고밀도에 유리한가 — 그리고 무엇을 희생하나

직렬의 이득: 셀마다 BL/SL에 직접 연결하는 콘택트(contact)를 두지 않아도 된다. string 하나에 SSL/GSL 콘택트 2개만 있으면 그 안의 수십~수백 셀이 콘택트를 공유한다. 콘택트는 면적을 많이 잡아먹는데, 그걸 셀 수십 개당 두 개로 줄이니 셀을 극단적으로 뻥뻥이 채울 수 있다. **NAND가 세상에서 가장 bit당 면적이 작은(=싼) 메모리인 비결이 이 직렬 공유다.**

직렬의 대가 — random access 희생: 직렬이라 특정 셀 하나를 읽으려면 같은 string의 나머지 셀을 전부 Vpass로 열어 통로를 만들어야 한다(§2.1). 즉 셀 하나에 독립적으로 즉시 접근할 수 없고, string·page 전체를 동원해야 한다. 그래서:

- random access(임의 주소 1바이트 즉시 읽기)가 느리고 비효율적 → DRAM처럼 working memory로 못 씀.
- 대신 큰 덩어리(page)를 한 번에 읽고 쓰는 데 최적 → storage로 완벽.
- erase가 block 단위인 것(§5)도 이 공유 구조의 연장선.

이게 §0의 "NAND = storage" 역할을 물리에서 다시 확인하는 지점이다. 직렬 공유로 밀도를 극대화한 대가로 random access를 포기했고, 그래서 working memory(DRAM)가 아니라 storage가 됐다. 구조 → 성질 → 역할이 하나의 사슬로 이어진다.

7. 2D의 한계와 3D V-NAND — 평면을 포기하고 위로

7.1 왜 2D가 막혔나 — 미세화의 막다른 골목

처음엔 NAND도 평면(2D)에서 셀을 작게 만드는 미세화로 밀도를 올렸다. 그런데 셀을 계속 줄이자 두 벽에 부딪혔다(★A7):

1. **셀 간 간섭(coupling)**: 셀이 가까워질수록 옆 셀의 전하가 내 Vth에 영향. FG 도체 그릇은 특히 심함(§1.3). Vth 분포가 흐트러져 0/1(나아가 다중 레벨) 구분이 어려워짐.
2. **저장 전자 수 감소**: 셀이 작아지면 저장층 부피가 줄어 가둘 수 있는 전자 수가 수십 개 수준까지 떨어진다. 전자 몇 개만 새어도 Vth가 크게 흔들려 신뢰성이 무너진다. "통계적으로 셀을 운영할 전자가 부족"해지는 한계.

평면에서 더 줄이면 신뢰성이 붕괴한다. 그래서 발상을 바꿨다: **셀을 더 줄이지 말고, 위로 쌓자.**

7.2 3D V-NAND의 발상 — Z축으로 도망가기

3D NAND(V-NAND)는 셀을 평면에서 뻥뻥이 줄이는 대신 **수직으로 적층**한다. 핵심 공정 그림:

1. 여러 게이트(WL) 층과 절연층을 번갈아 먼저 쌓는다 — 수백 층의 샌드위치.
2. 그 적층을 위에서 아래로 깊게 뚫어 channel hole(채널 구멍)을 낸다 — 하나의 깊은 우물.

3. 구멍 벽을 따라 터널 산화막 / 전하 저장층(CTF 질화막) / 채널을 차례로 두른다. 그러면 구멍 벽을 따라 수직으로 직렬 연결된 셀들의 string이 한 번에 만들어진다.

비유: 평면에 단층집을 다닥다닥 짓다 한계에 부딪히자, **고층 아파트(수직 적층)**로 전환한 것. 같은 땅(웨이퍼 면적)에 층(단)을 올려 용량을 키운다. 그리고 결정적으로, 위로 쌓으니 **각 셀(방)을 굳이 작게 안 만들어도 된다** — 오히려 셀 크기를 완화해 **전자 수·신뢰성을 회복**할 수 있다. "고밀도와 신뢰성을 동시에" 잡는 묘수.

여기서 §1.4가 닫힌다: channel hole 벽을 따라 얇은 막으로 두르는 구조에는 **얇고 결함에 강한 CTF**가 도체 FG보다 압도적으로 유리하다. 그래서 3D NAND $\approx$ CTF.

7.3 3D의 난제 — 위로 쌓을수록 어려워지는 것들

단수를 무한정 올릴 수 없는 이유(★A7 꼬리):

- **고종횡비(high aspect ratio) 식각**: 수백 층을 관통하는 깊고 가는 구멍을, 위아래 지름이 거의 같게 균일하게 뚫어야 한다. 너무 깊으면 아래로 갈수록 좁아지거나(테이퍼), 휘거나, 막힌다. 식각 난도가 단수에 따라 급격히 올라간다.
- **string stacking(스트링 적층)**: 한 번에 다 못 뚫으니 **여러 덩이로 나눠 쌓고 이어 붙인다**(예: 100여 단씩 쌓아 접합). 접합부 정렬·균일성이 새로운 과제.
- **적층 스트레스·휨(warping)**: 수백 층을 쌓으면 막 응력이 누적돼 웨이퍼가 휜다. 휨은 정렬·수율을 망친다.
- **string 저항 증가**: 직렬 셀이 많아질수록 string을 관통하는 전류 경로의 저항이 커져 read 속도·마진에 부담.
- **peri 면적 — CuA / hybrid bonding**: 주변 회로(peri)를 셀 어레이 아래로 넣거나(Cell-under-Array), 아예 peri를 별도 웨이퍼로 만들어 셀 웨이퍼에 본딩(hybrid bonding, CMOS-bonded-array)하는 식으로 면적 효율을 짜낸다.

업계는 현재 **200~300단대를 양산**하고 **400단+를 개발 중**이다(추정·변동 — 단수는 분기마다 갱신되니 면접에서 특정 숫자 단정은 피하고 "수백 단, 계속 상승" 정도로). SK하이닉스·삼성 모두 이 경쟁의 선두권에 있다.

브릿지(★A7): "단수 $\uparrow$ $\rightarrow$ page buffer·주변 제어 로직 복잡도 $\uparrow$ $\rightarrow$ 검증 부담 $\uparrow$." NAND가 위로 갈수록 Peri 디지털 로직과 그 검증·자동화 가치가 커진다. 셀 어레이의 물리는 공정의 몫이지만, 그 어레이를 제어하는 로직은 **네 영역**이다.

8. 비트 밀도 — SLC/MLC/TLC/QLC/PLC, Vth를 몇 조각으로 쪼개나

§1에서 "전자 가득 = Vth 높음 = 0, 비어 있음 = Vth 낮음 = 1"이라 했다. 그건 셀당 1비트(두 레벨)일 때다. **셀 하나의 Vth 축을 더 잘게 쪼개면, 한 셀에 여러 비트를 담을 수 있다.**

명칭	bit/cell	Vth 레벨 수	한 줄 성격
SLC (Single)	1	2	가장 빠르고·튼튼·고가. 캐시·산업용
MLC (Multi)	2	4	균형. (관용상 2bit를 MLC라 부름)
TLC (Triple)	3	8	현재 소비자 SSD 주류
QLC (Quad)	4	16	대용량·저가. 읽기 위주(아카이브)
PLC (Penta)	5	32	차세대 시도. 마진·신뢰성 난제 큼

원리: 저장 전자 수를 정밀히 조절해 Vth를 **2ⁿ개의 좁은 분포**로 나눠 배치한다. n비트면 2ⁿ개 레벨. read는 그 레벨들 사이에 여러 개의 판정 전압을 걸어 어느 구간인지 알아낸다.

trade-off의 정수 — bit/cell을 늘리면:

- 밀도↑·비용/bit↓ (좋다): 같은 셀에 더 많은 정보.
- 마진↓ (나쁘다): Vth 축을 더 잘게 쪼개니 레벨 간 간격이 좁아진다. 전자 몇 개의 누설로도 옆 레벨로 넘어가 오류 → raw BER(원시 오류율)↑.
- 속도↓: ISPP가 더 정밀해야 하니 program이 느리고(더 많은 step/verify), read도 판정 전압을 여러 번 걸어 느림.
- 수명↓: 좁은 마진은 oxide 열화·전하 손실에 더 취약 → P/E 수명 짧음(§9).
- ECC 부담↑: 높아진 raw BER을 강한 ECC(LDPC)로 메워야 함.

한 문장: "bit/cell↑ = Vth를 더 잘게 쪼개 밀도·원가는 좋아지지만, 마진·속도·수명이 나빠지고 ECC 부담이 커지는 정직한 trade-off." QLC는 그래서 읽기 위주 대용량 저장에 쓰고 쓰기 많은 데는 안 맞는다.

9. 신뢰성과 수명 — 셀은 닳고, 흔들리고, 새고, 그걸 로직이 덮는다

NAND 셀은 본질적으로 **불완전한 저장소**다. 쓰면 닳고(P/E 마모), 읽으면 흔들리고(disturb), 시간이 지나면 샌다(retention). 이 모든 불완전함을 컨트롤러의 디지털 알고리즘이 덮어 신뢰성 있는 스토리지로 변환하는 것 — 이게 NAND 설계의 본질이다. 여기가 네 영역과 가장 가까운 지점이다.

9.1 P/E cycle과 tunnel oxide 열화 — 수명의 근원

program·erase는 매번 FN 터널링으로 전자가 터널 산화막을 강제로 통과하는 일이다(§3·§4). 이 거친 통과가 산화막 격자에 미세한 손상(trap, 결함)을 누적시킨다. 통과 횟수가 쌓일수록 산화막이 닳아 — ① 전자가 잘 새고(retention 저하), ② Vth 제어가 부정확해지고, ③ 결국 셀이 신뢰 범위를 벗어난다.

그래서 NAND 수명은 P/E cycle(쓰기·지우기 반복 횟수)로 제한된다. 대략(추정·변동):

타입	P/E cycle 내구 (대략)
SLC	~10만 회
MLC	~3천 ~ 1만 회
TLC	~1천 ~ 3천 회
QLC	~수백 ~ 1천 회

bit/cell이 클수록 마진이 좁아 같은 oxide 열화에도 더 빨리 한계에 닿는다(§8과 연결). 읽기에는 거의 마모가 없다(전압이 약하고 터널링이 아니므로) — 다만 읽기는 disturb라는 다른 문제를 만든다(아래).

9.2 Read disturb — 읽기만 했는데 망가지는 역설

§2.1에서 심은 씨앗을 거둔다. 한 page를 읽으려면 같은 string의 **비선택 셀에 Vpass를 건다**(통로로 만들려고). Vpass는 program 전압만큼 세진 않지만 0은 아니다. 그래서 **약한 전하 주입**을 일으킨다. 한두 번은 무시할 수준이지만, **같은 영역을 수천·수만 번 반복해 읽으면** 비선택 셀들에 전하가 야금야금 쌓여 Vth가 슬슬 올라가고, 결국 bit가 뒤집힌다.

핵심: read disturb는 "읽힌 셀"이 아니라 "같은 string의 옆(비선택) 셀"이 피해자라는 점. 내가 A를 반복해 읽었는데 같은 string의 B가 망가진다. "반복 access가 인접 셀을 교란"하는 구조 — 이 패턴을 기억하라(§12에서 RowHammer와 대칭).

대응: ECC로 정정 + 특정 block의 누적 read 횟수를 컨트롤러가 세어, 임계 넘으면 데이터를 다른 곳으로 옮겨 다시 쓰는 read reclaim(read-disturb-aware refresh).

9.3 Program disturb — 쓰기 중 옆 셀 교란

program 중에도 비슷한 일이 있다. 한 셀에 Vpgm을 걸 때, 같은 WL이나 같은 BL을 공유하는 **인접 셀에 의도치 않은 약한 전계**가 걸려 전하가 조금 주입될 수 있다. read disturb가 읽기 반복 누적이라면, program disturb는 쓰기 시점에 인접 셀이 교란되는 것. 둘 다 ECC와 program inhibit·전압 최적화로 완화한다.

꼬리 정리(★D5): "program disturb = 쓰기 중 인접 셀 교란, read disturb = 읽기 반복 누적 교란. 둘 다 NAND string의 전압 공유 구조가 만든 부작용이고, ECC + 관리 로직으로 뚫는다."

9.4 Retention — 시간이 지나며 새는 전하

P/E를 안 해도, 그냥 시간이 흐르면 저장층 전자가 아주 천천히 새거나 재분포해 Vth가 흐트러진다(retention loss). oxide가 얇은 셀(P/E 많이 한 셀)·고온일수록 심하다. retention 저하 역시 ECC와 주기적 refresh(필요 시 재기록)로 관리한다.

9.5 Wear leveling — 고르게 닳게 하기

수명이 P/E cycle로 제한되니, 특정 block만 집중적으로 쓰면 그 block이 먼저 죽는다. 컨트롤러(FTL)는 write를 전체 block에 **고르게 분산**해 마모를 균등화한다. "자주 바뀌는 데이터를 한 곳에 몰지 말고 골고루" — 덕분에 디바이스 전체 수명이 늘어난다.

9.6 Bad block 관리 — 죽은 셀 격리

제조 시점부터 불량인 block(initial bad block)과, 쓰다가 수명이 다해 신뢰할 수 없게 된 block(runtime bad block)을 목록으로 관리하고 사용에서 배제한다. 데이터는 정상 block으로만 보내고, 불량은 매핑에서 제외.

9.7 ECC(LDPC) — 오류를 코드로 되돌리기

위 모든 메커니즘(마모·disturb·retention)이 만든 **bit 오류를 강력한 오류정정부호로 정정**한다. 현대 NAND는 주로 **LDPC(Low-Density Parity-Check)** 코드를 쓴다. 단순 해밍보다 정정 능력이 훨씬 강해, bit/cell↑로 높아진 raw BER을 감당한다.

- raw BER이 한계에 가까워지면 **read-retry**(판정 전압을 조금씩 옮겨 가며 다시 읽기), **voltage tracking**(셀 분포 변화를 추적해 판정점 보정) 같은 추가 기법으로 디코딩을 돕는다.
- QLC처럼 마진이 좁을수록 더 강한 LDPC·더 정교한 read 보정이 필요 → **컨트롤러 부담↑**.

이 절 전체의 메시지(★D4·D5·C7과 직결): NAND 셀은 닳고 흔들리고 샌다. 그 **물리적 불완전함을 wear leveling + read reclaim + bad block 관리 + LDPC ECC**라는 **디지털 알고리즘**이 덮어, 사용자에게는 안정적인 스토리지로 보이게 한다. "소자의 비신뢰성을 로직으로 신뢰성으로 변환" — 이 한 문장이 NAND 컨트롤러 설계의 정수이고, 네 검증·로직 경험이 바로 닿는 곳이다.

9-A. ★ Read·Program·Erase의 레이턴시와 쓰루풋 — "왜 읽기는 빠르고 쓰기·소거는 느린가"

면접에서 "read/program/erase를 레이턴시·쓰루풋 관점에서 설명하라"는 질문이 들어오면, **세 동작의 비대칭(asymmetry)** 과 **레이턴시 vs 쓰루풋의 구분**, 두 축으로 답해야 한다.

9-A.1 세 동작의 레이턴시 — 크기의 순서가 핵심

동작	대략적 레이턴시(통상·변동)	무엇을 기다리나
Read (tR)	수십 μ s급 (가장 빠름)	WL에 read 전압 인가 → 한 번의 sensing(셀이 켜지나/꺼지나 판정). TLC는 여러 판정점을 봐야 해 SLC보다 김
Program (tPROG)	수백 μ s급 (read의 ~10배)	ISPP: "약한 펄스 → verify → 더 센 펄스 → verify"를 여러 번 반복 해 Vth를 목표 창에 정밀하게 밀어 넣음. TLC/QLC는 분포를 더 촘촘히 나눠야 해 반복이 더 많음
Erase (tBERS)	수 ms급 (가장 느림)	블록 전체 의 전자를 한꺼번에 FN 터널링으로 토해내게 함 + verify. 다루는 구조가 가장 크고 전압이 높아 가장 오래 걸림

핵심 한 줄: 읽기 $\ll$ 쓰기 $<$ 소거 (대략 μ s $\ll$ 수백 μ s $<$ ms). 이 부등호의 **이유**까지 말해야 만점.

왜 이 순서인가 — 한 문장씩: - 읽기가 빠른 이유: 전하를 건드리지 않고 한 번 "켜지나?"만 보면 된다(비파괴·단일 sensing). - 쓰기가 느린 이유: 전하를 절연막 안에 *정밀한 양*으로 가둬야 하는데, 한 방에 못 넣어 ISPP로 조금씩 밀어 넣고 매번 확인한다(반복). 멀티레벨일수록 창이 좁아 더 느려진다. - 소거가 가장 느린 이유: page 하나가 아니라 **블록 전체**(수백~수천 WL)를 동시에 비워야 한다(5장의 "erase는 block 단위" 귀결). 큰 덩어리 + 높은 전압.

9-A.2 레이턴시 vs 쓰루풋 — 같은 말이 아니다

- 레이턴시(latency) = 한 동작이 끝나는 데 걸리는 시간(위 표).
- 쓰루풋(throughput) = 단위 시간당 옮기는 총 데이터량(MB/s, IOPS).

NAND는 개별 레이턴시가 높음에도 쓰루풋은 높다. 비결은 병렬성과 파이프라이닝이다: - 큰 page: 한 번의 read/program이 16KB급 page를 통째로 처리 → 1회 동작에 많은 바이트. - multi-plane: 한 die 안의 여러 plane을 동시에 read/program. - multi-die / multi-channel interleaving: 컨트롤러가 여러 die·채널에 동작을 겹쳐 발행 → 한 die가 program으로 바쁜 동안 다른 die가 데이터 전송. 긴 tPROG를 숨긴다(hide).

즉 "레이턴시는 물리로 정해지지만, 쓰루풋은 아키텍처(병렬·인터리빙)로 만든다." 이 한 문장이 디지털 설계자/컨트롤러 관점의 핵심이다.

9-A.3 비대칭이 시스템에 강제하는 것

- erase-before-write: 이미 쓴 자리에 덮어쓰기 불가 → 새 블록에 쓰고 옛 블록은 나중에 소거. 그래서 GC(garbage collection)·over-provisioning·wear leveling(§9-§10)이 필연.
- read-heavy 친화: 읽기가 싸므로 read-dominant 워크로드에 유리. 쓰기는 버퍼링·정렬로 tPROG를 분산.
- 랜덤 vs 순차: 순차 쓰기는 page/plane을 꽉 채워 쓰루풋↑, 랜덤 소량 쓰기는 GC·write amplification으로 불리.

9-A.4 면접 골격 (30초)

"세 동작은 레이턴시가 읽기 ≪ 쓰기 < 소거 순입니다. 읽기는 전하를 건드리지 않고 한 번 판정만 하니 수십 μ s로 빠르고, 쓰기는 ISPP로 V_{th} 를 조금씩 밀어 넣고 매번 verify해 수백 μ s, 소거는 블록 전체를 비워야 해 수 ms로 가장 느립니다. 다만 개별 레이턴시가 높아도 multi-plane·multi-die 인터리빙으로 병렬화해 쓰루풋은 높게 만듭니다 — 레이턴시는 물리, 쓰루풋은 아키텍처입니다. 이 비대칭 때문에 erase-before-write·GC가 생기고, 그게 컨트롤러(FTL) 설계의 핵심입니다." (수치는 변동, "크기의 순서"가 본질)

🧠 설계 브릿지: 긴 tPROG/tBERS를 인터리빙으로 숨기는 건 디지털 설계의 파이프라이닝·자원 분할 (throughput hiding) 과 같은 사고다. 컨트롤러의 동작 스케줄링·다중 채널 arbitration은 전형적 RTL 설계·검증 대상 — 내 영역.

10. 컨트롤러와 FTL — NAND를 "쓸 수 있는 디스크"로 만드는 디지털 로직

지금까지 본 모든 제약 — 제자리 덮어쓰기 불가(§5), 마모(§9), disturb(§9), bad block(§9) — 을 호스트(OS)가 직접 신경 쓰면 NAND는 못 쓴다. 그래서 그 사이에 컨트롤러가 있고, 그 핵심 소프트웨어/로직이 FTL(Flash

Translation Layer)이다. FTL은 NAND의 거친 물리를 감추고, 호스트에게는 "아무 주소나 자유롭게 읽고 쓰는
평범한 디스크"라는 깔끔한 추상을 제공한다.

FTL의 3대 임무:

- 1. **주소 매핑(L2P, Logical-to-Physical)**: 호스트가 말하는 논리 주소(LBA)를 NAND의 물리 위치(block/page)로 변환. 제자리 덮어쓰기가 안 되니, 같은 논리 주소라도 쓸 때마다 다른 물리 위치로 가고, 매핑 테이블이 그걸 추적한다. "옛 데이터는 invalid 표시, 새 위치로 매핑 갱신."
- 2. **Garbage Collection(GC)**: invalid page가 흩어진 block들에서 valid page만 모아 새 block으로 옮기고, 빈 block을 erase해 재사용. §5.4의 write amplification·over-provisioning이 여기서 작동.
- 3. **Wear leveling**: §9.5. 마모를 전 block에 고르게 분산.

여기에 read reclaim, bad block 관리, ECC 엔진, 그리고 호스트 인터페이스(NVMe 등) 제어가 더해진다.

이 모든 것이 디지털이다. 매핑 테이블 관리, GC 스케줄러, wear leveling 알고리즘, read-disturb 카운터와 임계 비교 FSM, ECC 인코더/디코더 RTL, 인터페이스 프로토콜 — 전형적인 **디지털 RTL 블록 + 펌웨어**다. ECC 디코더의 타이밍, 인터페이스의 클럭 도메인 크로싱(CDC), 컨트롤러 SoC의 STA·DFT/MBIST — 네가 4년간 sign-off·검증한 그 일이 여기에 그대로 있다.

11. ★ DRAM vs NAND 대조표 — 기출 대비 핵심

면접 ★A1의 정답 골격. 셀 구조 나열이 아니라 "역할이 달라 물리를 다르게 골랐다"는 인과를 표 위에 얹어 말하라.

축	DRAM	NAND Flash
휘발성	휘발성 (전원 끄면 소실)	비휘발성 (전원 꺼도 유지)
셀 구조	1T1C (트랜지스터1 + 커패시터1)	전하 저장 트랜지스터 (FG/CTF), string 직렬
저장 원리	커패시터 전하의 유무	절연막에 갇힌 전하 → Vth 이동
접근 단위	random access (byte/word)	read/program= page , erase= block
제자리 갱신	가능 (덮어쓰기 자유)	불가 (새 page에 쓰고 block erase)
속도	매우 빠름 (ns급)	느림 (program/erase 특히, ISPP·터널링)
수명	사실상 무제한 (단 refresh 필요)	P/E cycle 제한 (마모)
유지 비용	refresh (누설 보충)	refresh 불필요, 대신 ECC·wear leveling·GC
밀도·원가	중간	최고 밀도·최저 비용/bit
용도	working memory (메인 메모리)	storage (영구 저장)

두 메모리는 "전하를 담는다"는 같은 행위를 하되, **그릇의 절연 품질과 셀 연결 방식을 정반대로 설계해** — DRAM은 빠르고 휘발적인 working memory로, NAND는 느리지만 비휘발·고밀도인 storage로 — 갈라졌다.

12. 면접 연결 — 30초/1분 골격 + 삼성 로직 → NAND 브릿지

12.1 ★ "NAND를 설명해보세요" (30초 / 1분)

30초 골격

"NAND는 트랜지스터 게이트 아래 절연막에 전자를 가둬, 그 전하가 문턱전압 V_{th} 를 옮기는 걸로 0/1을 저장하는 비휘발성 메모리입니다. 전하 주입(program)·방출(erase)에 FN 터널링을 쓰고, 셀을 string으로 직렬 연결해 콘택트를 공유함으로써 세상에서 가장 뽀빠한 — 비트당 가장 싼 — 메모리가 됩니다. 그 대가로 random access를 포기해 storage 용도로 쓰이고, 평면 한계 때문에 셀을 수직 적층한 3D V-NAND로 갔습니다."

1분 골격(위 + 단위·신뢰성·컨트롤러)

"...구조상 read/program은 워드라인 단위라 page로, erase는 셀이 공유하는 well에 전압을 걸어야 해서 block 단위로 일어납니다. 이 비대칭 때문에 제자리 덮어쓰기가 안 되고, 그래서 컨트롤러의 FTL이 주소 매핑·garbage collection·wear leveling로 이를 덮습니다. 셀은 P/E로 마모되고 read disturb·retention 저하 같은 약점이 있는데, 이를 LDPC ECC와 read reclaim 같은 디지털 알고리즘으로 신뢰성 있는 스토리지로 변환합니다. 셀 어레이는 소자·공정 영역이지만, 그 어레이를 제어하는 Peri와 컨트롤러는 제가 해온 RTL·검증·sign-off가 그대로 닿는 디지털 영역입니다."

12.2 ★ "왜 3D(V-NAND)로 갔나요?" (30초)

"2D 평면 미세화가 셀 간 간섭과 저장 전자 수 감소로 신뢰성 한계에 부딪혔습니다. 셀을 더 줄이는 대신 수직으로 적층하면, 같은 면적에 용량을 키우면서 오히려 셀 크기를 완화해 신뢰성을 회복할 수 있습니다. 한번에 깊은 channel hole을 뚫어 그 벽을 따라 string을 만들고, 얇고 결함에 강한 charge trap 구조가 여기에 잘 맞습니다. 다만 고종횡비 식각·string stacking·적층 힘이라는 새 난제가 생겨 단수를 올릴수록 어렵습니다."

12.3 ★ Read disturb ↔ RowHammer 대칭 — 강점 어필 포인트

이건 너의 디지털·검증 강점을 메모리 디바이스에 꽂는 최고의 다리다.

"NAND의 read disturb는 한 셀을 반복해 읽을 때 같은 string의 비선택 셀이 V_{pass} 로 교란돼 bit가 뒤집히는 현상이고, DRAM의 RowHammer는 한 row를 반복 access할 때 인접 row 셀이 교란돼 bit flip이 나는 현상입니다. 둘 다 '반복 access가 인접 셀을 교란한다'는 같은 구조이고, 대응도 같은 패턴입니다 — access 횟수를 모니터링하는 카운터/FSM + 임계 초과 시 데이터 재배치/추가 refresh(read reclaim / TRR). 이건 정확히 제가 해온 디지털 제어 로직 + 타이밍 검증의 문제입니다. 소자의 물리적 약점을 디지털 모니터링·완화 로직으로 덮고, 그 로직을 검증하는 일 — 메모리든 로직이든 검증의 문제는 같습니다."

12.4 삼성 로직 → NAND Peri/컨트롤러 브릿지 (총괄)

- **너의 좌표**: NAND의 절반은 셀 어레이(소자·공정), 절반은 Peri + 컨트롤러(디지털 RTL·펌웨어)다. 후자는 네 4년 경험이 그대로 적용된다.
- **CDC/RDC** → 컨트롤러의 호스트 인터페이스(NVMe)·내부 코어·NAND I/O가 다중 클럭/리셋 도메인. CDC/RDC sign-off가 그대로 필요.
- **STA** → 고속 NAND 인터페이스(ONFI/Toggle)·컨트롤러 SoC의 timing closure. read/program latency의 타이밍 마진.
- **DFT/MBIST/ECC** → 컨트롤러 내 ECC 인코더/디코더, 내장 메모리 MBIST·redundancy. 수율 직결.
- **검증 자동화/AI-EDA** → 단위·bit/cell↑로 Peri·컨트롤러 복잡도가 폭증하는 시대에, 네 검증 TAT 단축이 직접 가치.
- **한 문장 명제**: "NAND를 몰라서가 아니라, NAND의 디지털 절반 — Peri와 컨트롤러 — 이 곧 제가 하던 일입니다. 셀의 물리는 빠르게 배우고, 그 물리적 약점을 로직으로 덮는 일은 제 강점입니다."

13. 스스로 점검 — 막히면 위로 돌아가 다시 읽어라

1. NAND가 **비휘발성**인 **물리적** 이유를 절연막·전하·터널링으로 한 호흡에 설명할 수 있는가? (§0·§1.2)
2. 저장된 전하가 어떻게 **Vth**를 **옮기고**, 그 Vth가 어떻게 0/1이 되는지, 그리고 read가 왜 "켜지냐/꺼지냐 판정"인지 말할 수 있는가? (§1.1·§2)
3. **FG vs CTF**를 "연속(도체) vs 이산(절연체)"으로 환원하고, 거기서 셀 간섭·결함 내성·3D 적합성을 **인과**로 끌어낼 수 있는가? (§1.3·§1.4)
4. 왜 **program**은 **1→0 한 방향뿐**이고, 그게 왜 "제자리 덮어쓰기 불가 → FTL 필요"로 이어지는지 설명할 수 있는가? (§3.2·§5.4)
5. **ISPP**가 왜 **필요한지**(한 방에 못 쓰는 이유)와 program↔verify 루프를 다트 비유 없이도 말할 수 있는가? (§3.3)
6. ★ **read/program=page, erase=block**의 이유를, "WL에 전압 vs 공유 well에 전압"이라는 구조로 끝까지 설명할 수 있는가? (§5)
7. **NAND string**의 **직렬 공유**가 어떻게 고밀도를 만들고 무엇(random access)을 희생하는지, 그게 왜 NAND를 storage로 만드는지 사슬로 이을 수 있는가? (§6)
8. **2D 한계**(간섭·전자 수) → **3D 적층** → **CTF 적합** → **3D 난제**(고층황비 식각·string stacking·휨)를 한 흐름으로 말할 수 있는가? (§7)
9. **bit/cell↑의 trade-off** 5종(밀도↑·마진↓·속도↓·수명↓·ECC↑)을 "Vth를 더 잘게 쪼갬다"는 한 원리에서 모두 끌어낼 수 있는가? (§8)
10. **P/E 마모·read disturb·program disturb·retention**의 메커니즘을 각각 구분하고, 그것들을 **wear leveling·read reclaim·LDPC ECC**가 어떻게 덮는지 말할 수 있는가? (§9)
11. ★ **read disturb ↔ RowHammer 대칭**을 "반복 access가 인접 셀을 교란 → 모니터링 카운터 + 재배치/refresh"로 묶고, 그게 왜 **네 검증 강점**인지 한 문장으로 연결할 수 있는가? (§9.2·§12.3)
12. ★ **DRAM vs NAND**를 표가 아니라 "역할이 달라 물리를 정반대로 골랐다"는 인과로 30초 안에 말할 수 있는가? (§11)
13. "NAND를 설명하라"에 **30초·1분 두 버전**을 막힘없이 구술하고, 끝에 **Peri/컨트롤러 브릿지 한 문장**을 붙일 수 있는가? (§12)

13개 중 막히는 게 있으면 그 절로 돌아가라. 외우지 말고, "왜?"를 한 번 더 물어 인과의 사슬이 끊긴 곳을 찾아라. NAND는 단 하나의 씨앗 — *절연막에 갇힌 전하가 V_{th} 를 옮긴다* — 에서 전부 자라난다.